

OpenSABR Build Guide

Latest Revision: 2026-08-10

This guide explains how to build OpenSABR (Open-source System for Auditory Brainstem Response). Further information is available in the [BioRxiv preprint](#) and the accompanying [GitHub repository](#).

Note on previous versions

For all experiments in the paper, we used a prototype version of OpenSABR with a different enclosure and interface PCB. We recommend that all new users follow the build guide for the current version of the system, described in this document. However, for reproducibility and record-keeping purposes only, the old build guide for the prototype version is available in the [Supplemental Materials of the BioRxiv preprint](#) and the old designs are available in the [‘prototype interface pcb’ folder on our github](#). Again, new users are encouraged to build the updated version described below.

Overview of the system

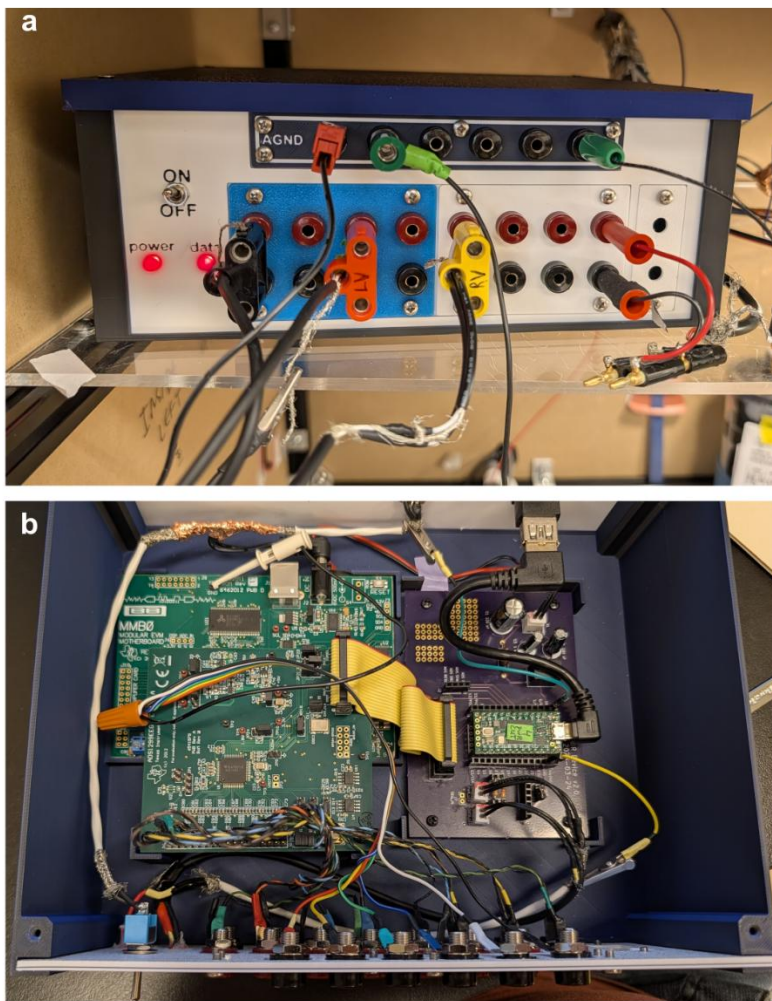


Figure 1. Hardware. **a)** Front panel of the enclosure. **b)** Inside of the enclosure with the top removed. Daughterboard and motherboard (green) on left side and interface PCB (purple) on right side, connected with a ribbon cable (yellow).

Main components and availability: OpenSABR's main electronics are Texas Instruments' ADS1299 development kit (consisting of a MMB0 motherboard and ADS199 daughterboard), a Teensy 4.0 microcontroller, and a custom interface printed circuit board (PCB) all housed in a 3D printed enclosure (**Fig. 1**). You will also need some custom cables, electrodes, and a Linux PC. All the components are commercially available and we provide a full [Bill of Materials](#).

Interface PCB. The interface PCB can be ordered [from our project shared on OSH Park](#), or you can get the PCB designs (**Fig. 2**) from our [Github's 'hardware designs' folder](#) and order the board from your preferred PCB manufacturer. We provide the files for the 3D printed enclosure and custom board spacers in the '[3D print designs](#)' section of our github.

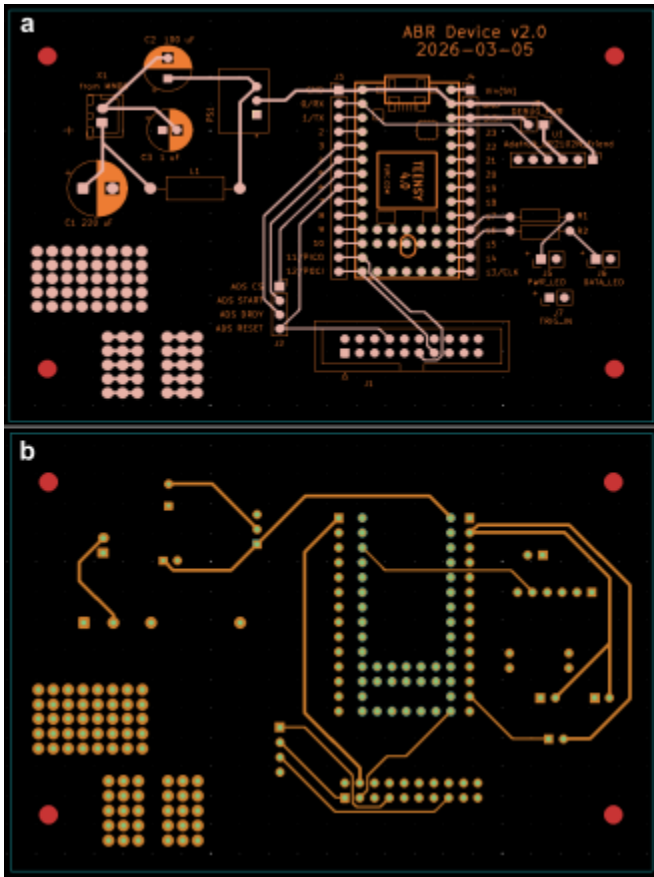


Figure 2. Layout of interface PCB. a) Front. b) Back.

Power: OpenSABR uses the MMB0 motherboard to provide power to the ADS1299 daughterboard and interface PCB. The interface PCB houses a voltage regulator to convert the MMB0's 9V power into the 5V power required by the Teensy.

Data and control signals: Raw voltage is collected at the electrodes, recorded by the ADS1299 daughterboard, and sent to the Teensy using 4-wire SPI protocol. The Teensy collects this data as packets, adds headers, and sends the packets to the desktop computer's GUI over USB (**Fig. 3**).

The Teensy controls the 'power' and 'data' indicator LEDs on the front panel of the enclosure. The 'data' indicator blinks when a recording is in process. It may stay on after the recording has ended, but this does not mean data is still recording.

The Teensy uses its TX and RX lines to send debug information to the optional USB-to-serial converter, Adafruit's CP2102N Friend. The CP2102N Friend uses a second USB cable to send data to the computer, which can be displayed in a serial monitor program such as minicom.

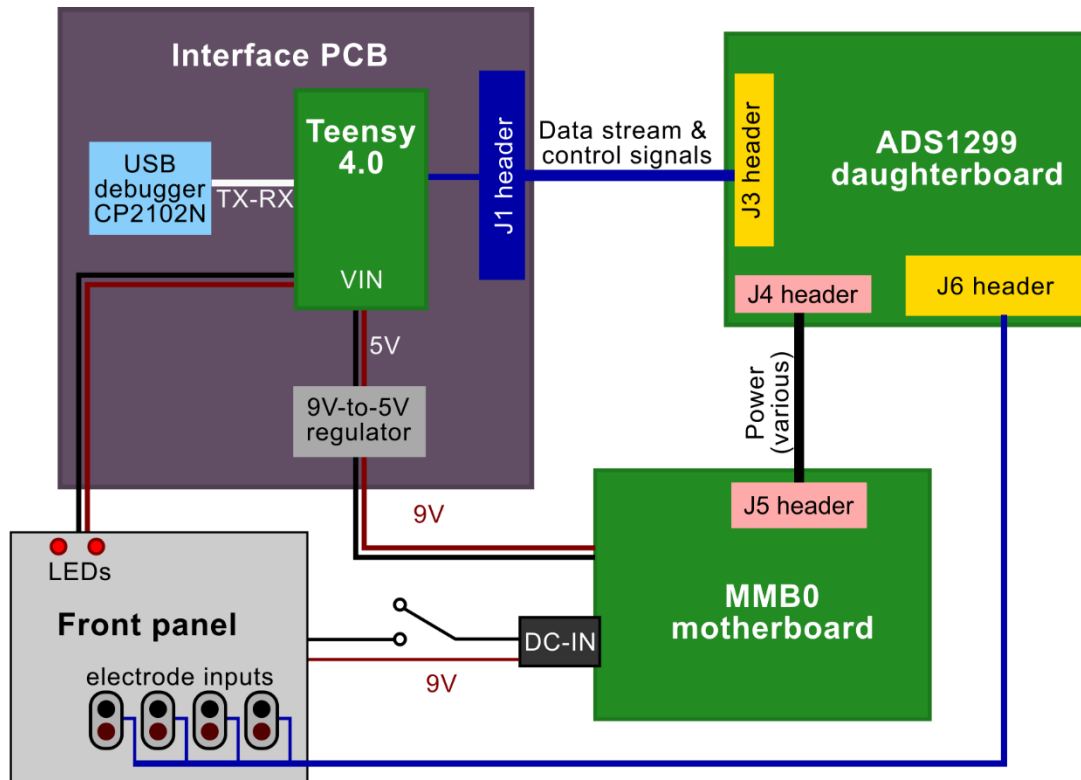


Figure 3. Overall schematic of the hardware inside the enclosure. The interface PCB (purple; upper left) houses the Teensy, the USB debugger, a power regulator, and a J6 header connected to the ADS1299 daughterboard. The ADS1299 daughterboard (green; upper right) houses the ADS1299 itself. The MMB0 motherboard (green; lower right) provides power. The front panel (gray; lower left) provides banana jacks for signal inputs and status indicator LEDs.

Physical enclosure

We designed a 3D printable enclosure for the device, which are the ‘Compact ABR enclosure’ files in [3D print designs folder of our github](#). The main body holds the interface PCB and MMB0 motherboard. The rear panel has holes for a power jack and feed-through USB adapter. The front panel (**Fig. 4**) has holes for a power switch, power and data indicator LEDs, and four slots to screw on modular faceplates:

- a long top faceplate with 8 banana jacks for grounding
- two faceplates, each with four channels of banana jacks for electrodes
- a single-channel faceplate on the far right for a TRIG IN channel

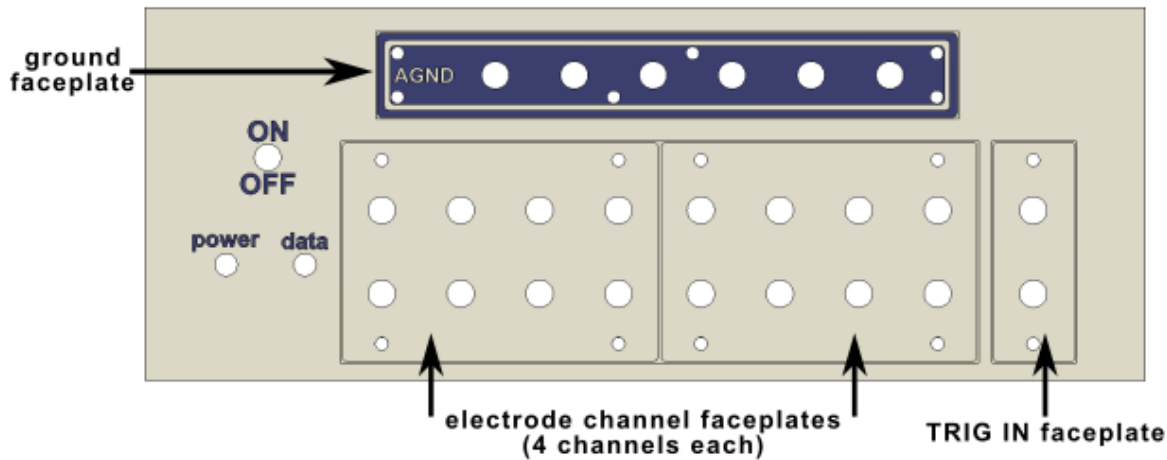


Figure 4. Design of front panel with modular faceplates labeled. This design allows you to print colored text without requiring a multi-color printer. Since the text is embossed, use your preferred slicing software to insert a pause after it's printed the bottom of the embossed text. Swap to the filament color you want for the main enclosure and resume the print.

Populating the interface PCB

First, solder all components to the interface PCB (**Fig. 5a**):

- A 2-pin JST-XH header on the X1 header for power input
- 2-pin JST-XH headers on the J5, J6, and J7 for the LEDs and optional TRIG_IN input
- R1 and R2 resistors for the power and data LEDs respectively (use appropriate value for your LEDs)
- A 9-to-5V step-down voltage regulator in PS1
- A 220 μ F capacitor in C1
- A 100 μ F capacitor in C2
- A 1 μ F capacitor in C3
- A 10 μ H inductor in L1
- Standard 2.54mm pitch female headers in the footprints provided
- A 2-pin 2.54mm-pitch male header in J8 (to power the USB debugger from Teensy power, add a jumper to short these two pins)

Make 2 cables between the power and data indicator LEDs to plug into the J5 and J6 connectors respectively (**Fig. 5b**).

The bottom left corner of the interface PCB has through-holes, which may be helpful for prototyping any add-ons or edits to the device. The lower rows of through-holes are shorted in breadboard-style rows, so see the PCB layout before using these (**Fig. 2**).

Screw the interface PCB into the enclosure base at the 4 corners using M3 screws. The holes are slightly under-sized so the screws should bite into the plastic and form threads.

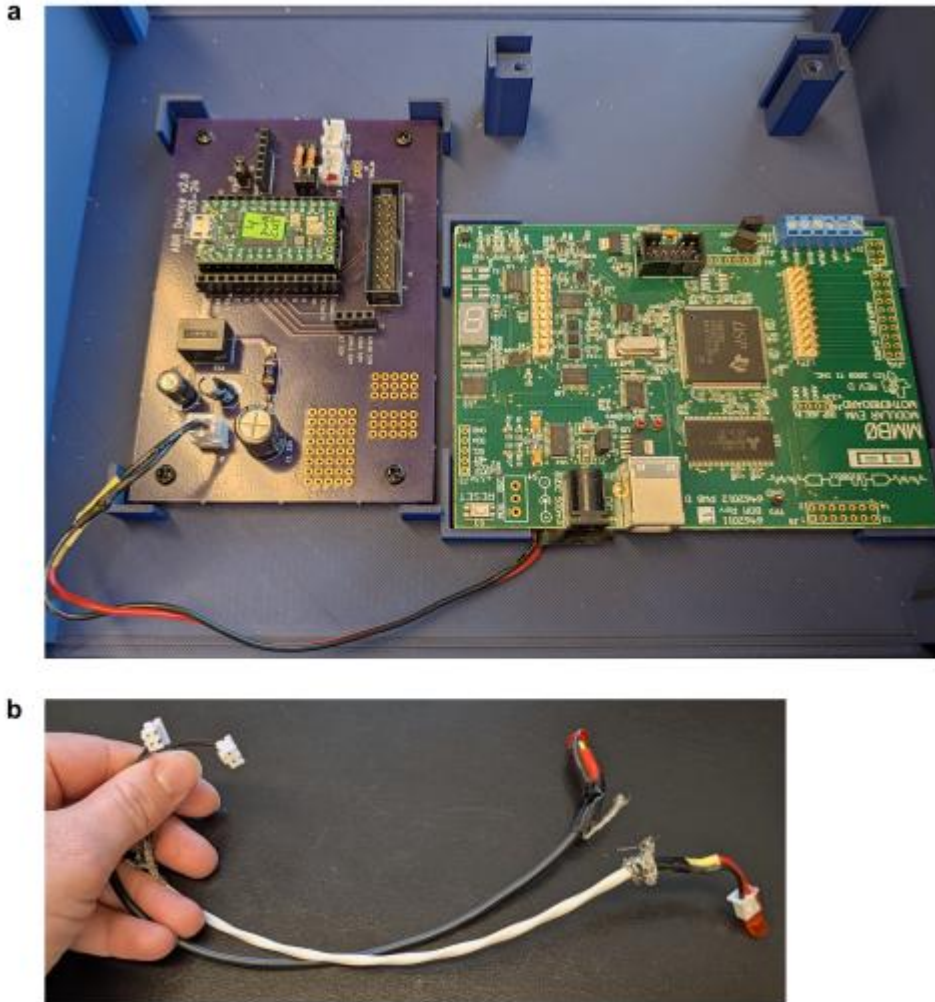


Figure 5. a) Base of the enclosure. Purple interface PCB (left) with all components soldered except for TRIG IN jack which we did not use. Green MMB0 motherboard (right) with power cable to the interface PCB. **b)** Cables for the power and data indicator LEDs. LEDs can be soldered directly to the wire, or the plugged into a female connector at the end of the wire.

Other devices on the interface PCB

USB debugger. The interface PCB's U1 connector is for an optional serial-to-USB debugger (Adafruit CP2102N Friend). Its TX and RX pins connected to Teensy pins 0 and 1 respectively. Shorting the pins of the J8 DEBUG_PWR header will connect the debugger's power to the 3.3V power from the Teensy, but the debugger typically is powered by its own USB cable so this is not necessary.

Teensy 4.0. Normally the Teensy would be powered by the USB connection, but we wanted it to be powered instead by the MMB0 motherboard so that it could be powered off and on by the front panel. To prevent contention on these power lines, it is necessary to cut the trace between the VIN and VUSB pads on the back of the Teensy (**Fig 6a**). Then add male headers to the Teensy's through-holes and plug it into the female headers on the interface PCB.

Setting up the MMB0 motherboard

Solder a power cable to the positive and ground leads of the MMB0's DC-IN jack (**Fig 6b**). We recommend using hot glue or electrical tape to provide mechanical stability here, because wire-to-board soldering puts

stress on the wire near the solder joint. Terminate the other end of this cable with a JST-XH connector and plug it into the interface PCB's X1 power connector.

Verify that on the MMB0 board jumper J12 is connected, J13A is not connected, and jumper J13B is connected. At this point, it should look like the picture in **Figure 5a**.

We made an adapter cable to insert a power switch into the circuit (**Fig. 6c**). The input jack (Same Sky PJ-011B) is mounted to the back panel, and the front panel's power switch interrupts the circuit before the right-angle plug is plugged into the MMB0's DC-IN jack. When soldering to the input jack, note that there are three pins and only two will provide power (**Fig. 6d**). The third pin is designed to provide power from a battery backup and will automatically disconnect when anything is plugged into the jack, so be sure to use the correct pin when soldering this cable.

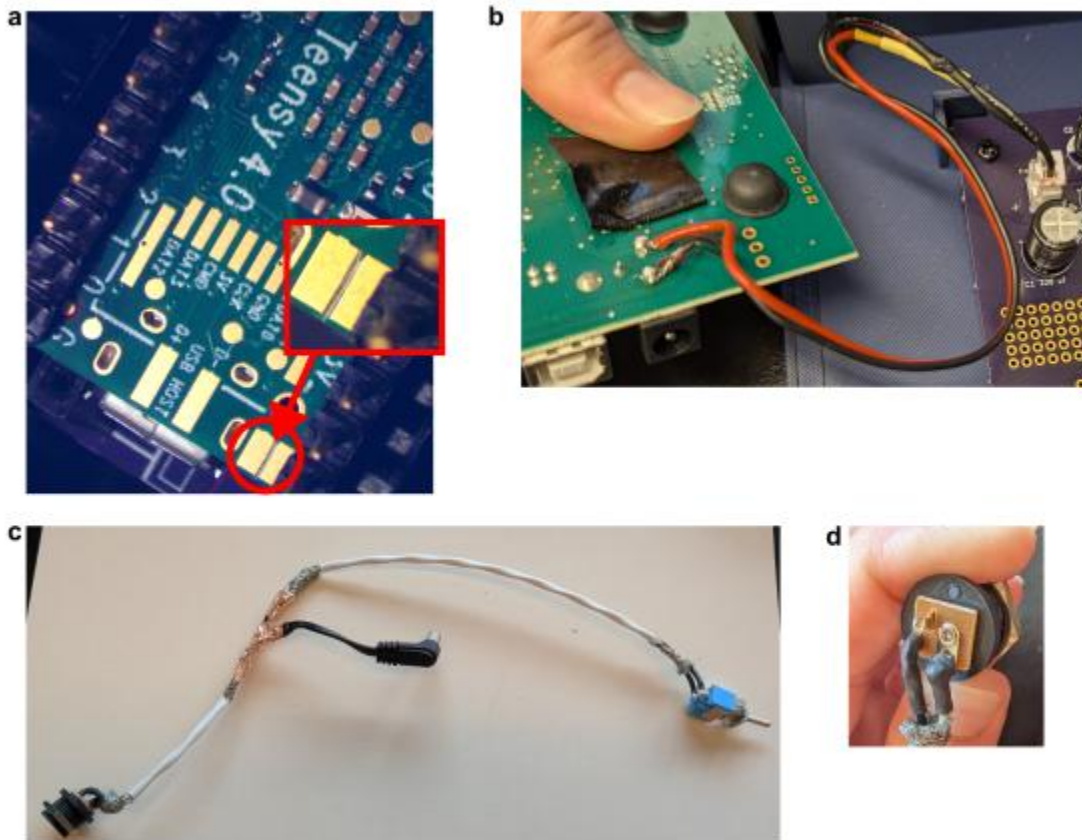


Figure 6. Powering OpenSABR. **a)** The cut trace between the VIN and VUSB pads on the back of the Teensy board. **b)** Power cable to the interface PCB, soldered to the bottom of the MMB0's DC-IN jack. **c)** Power cable adapter which inserts a power switch for the device before the MMB0's DC-IN jack. **d)** Close-up of the power input jack. The third unused pin is not connected and will not provide power.

Adding the ADS1299 daughterboard

The ADS1299 daughterboard is typically stacked on top of the MMB0 motherboard with three headers: the MMB0 motherboard labels these as J4 and J7 for data, and J5 for power. For OpenSABR we use the MMB0 to provide power without sending any data between the MMB0 and daughterboard.

Print two of the '2x10 male pin header spacer' from [the 3D print designs](#) in our Github. Slide these over the J4 and J7 headers on the MMB0 (**Fig. 7a**), where they can provide mechanical connection without electrical connection. Use a 2x5 stacking header to connect the MMB0's J5 connector to the daughterboard's J4

connector. Once the boards are stacked, use two M3 screws to fasten the daughterboard to the posts on the enclosure's base.

OpenSABR uses almost all of the factory default settings on the ADS1299 daughterboard, which you can verify by checking Table 1 of the development kit's [datasheet here](#). The exception is the jumpers JP2 and JP20 on the upper right of the board, which should be configured as shown in **Figure 7b** to use the +/- 2.5V bipolar power supply configuration. The board's white markings use a notched corner to mark pin 1, and these jumpers are both numbered right to left. JP2 should have pins 1-2 shorted and JP20 should have pins 2-3 shorted.

Use the 2x10 ribbon cable to connect the daughterboard's J3 header to the interface PCB's J1 header. Make sure the cable is plugged in the right orientation (**Fig. 7c**) with the daughterboard J3's CLKSEL pin connected to pin 1 on the interface PCB's J1 pin 1. **Table 1** lists the pinout for the daughterboard's J3 connector and the corresponding pin assignments on the Teensy, as well as other important Teensy pin assignments.

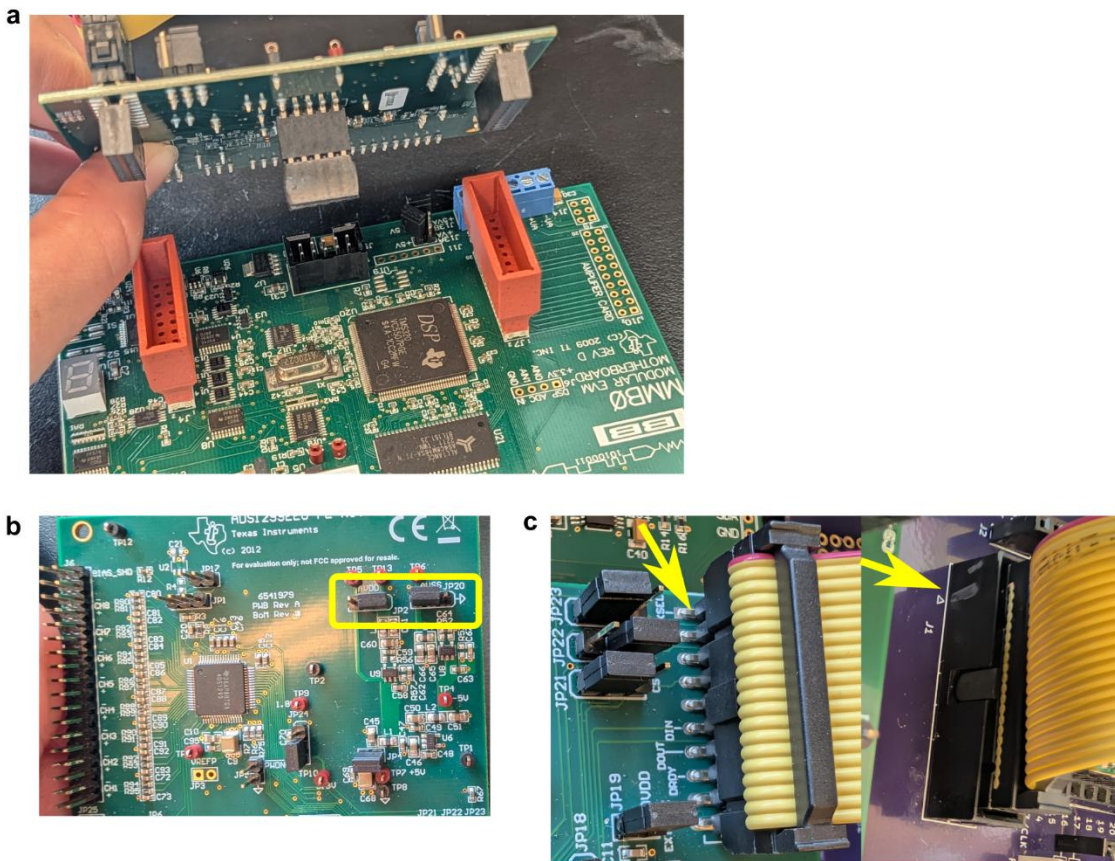


Figure 7. Daughterboard setup. **a)** Connectors between MMB0 motherboard and ADS1299 daughterboard. The MMB0's J4 and J7 headers have 3D printed hollow spacers. The MMB0's J5 header was connected to the daughterboard's J4 header using stacking headers. **b)** The daughterboard with JP2 and JP20 outlined in yellow. **c)** The correct orientation of the ribbon cable on the daughterboard (left) and interface PCB (right). Yellow arrowheads designate pin 1 on the connectors.

ADS daughterboard J3 (2x10 header)	
J3 pin number	Function
1	START
2	N/C
3	CLK
4	N/C
5	N/C
6	N/C
7	CS
8	RESET
9	N/C
10	GND
11	DIN
12	N/C
13	DOUT
14	N/C
15	DRDY
16	N/C
17	N/C
18	N/C
19	N/C
20	N/C

Teensy pin assignments	
Pin number	Function
0	RX1 (to debug's TX)
1	TX1 (to debug's RX)
3	DRDY
4	START
5	CS
6	RESET
11	MOSI
12	MISO
13	CLK
14	TRIG IN (Front panel)
15	DATA LED
16	POWER LED
GND	GND
VIN	5V from regulator

Table 1. Pinout of ADS1299 daughterboard's J3 header and Teensy GPIO pins. The left table lists the pins in the J3 header on the ADS daughterboard and corresponding J1 header on the interface PCB. The right table lists the Teensy's pin assignments and other connections.

Assembling front and back panels

Mount the power switch in the front panel as shown in **Figure 8a**, making sure that the orientation of the switch matches the orientation of the ON/OFF label on the panel. Push the LEDs into the 'power' and 'data' holes and use hot glue or tape to secure them.

Mount the power jack and USB adapter in the back panel as shown in **Figure 8b**. The USB adapter (CLIFF Electronics CP30205NM3B) uses M3 screws.

Slide the front and back panels into the grooves on the enclosure base. Use M3 screws to attach the AGND faceplate to the front panel.

Plug the power cable's right-angle plug into the MMB0 motherboard's DC-IN jack. Use the right-angle USB-micro-to-USB-A cable to connect the Teeny 4.0 to the USB adapter. Use short alligator-to-M-pin jumpers to connect any cable shielding to the Teensy's ground pins. At this point, it should look like **Figure 8c**.



Figure 8. **a)** Front panel with power switch and LEDs mounted. **b)** Back panel with USB adapter and power jack mounted. **c)** OpenSABR with front and back panels installed. Red arrow indicates grounding clip connecting the power cable shielding to the Teensy's ground.

Assembling the electrode and ground inputs

Each electrode input is a pair of wires soldered to the solder lug of one red and one black banana jack, terminating in a 2-pin standard 2.54mm pitch female header (**Fig. 9a**). Make 8 of these (9 if you intend to use the TRIG-IN jack).

Mount the banana jacks on the electrode channel faceplates as shown in **Figure 9a**. Feed the cables through the front panel and plug them into the daughterboard's J6 header as shown in **Figure 9b**, using only the even numbered pins (right side). Follow the white print on the daughterboard, with channel one's positive input at the bottom and channel 8's negative input closest to the top. The top pins next to 'BIAS_SHD' are reserved for a bias configuration which we did not use. Screw the electrode channel faceplates to the front panel with M3 screws.

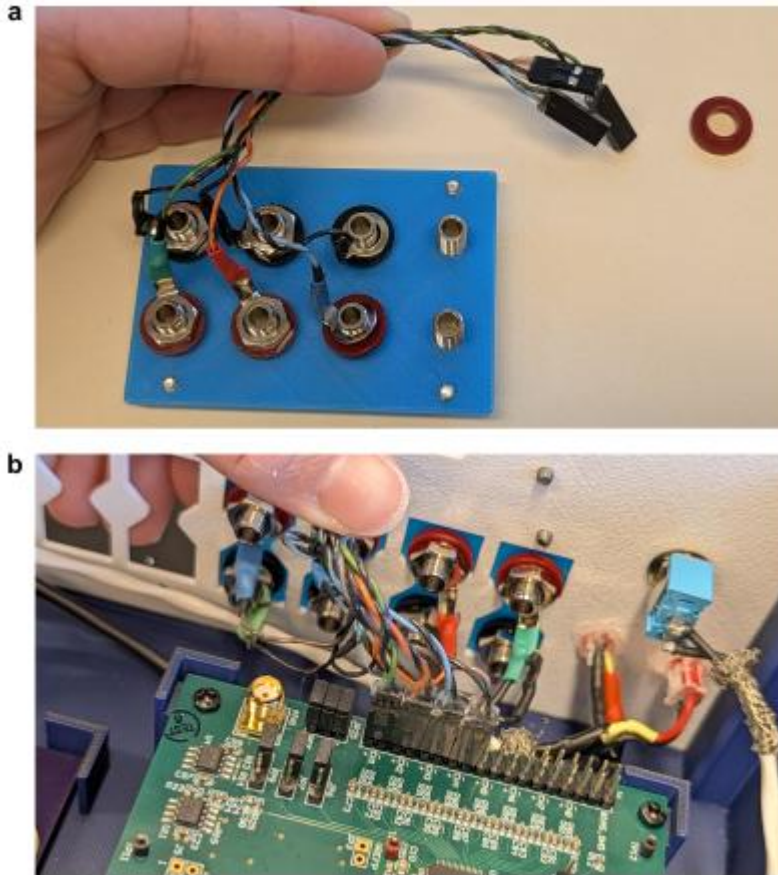


Figure 9. a) Electrode input jacks with cables, mounted to the electrode channels faceplate. **b)** Electrode channels faceplate mounted to the front panel, with electrode cables plugged into the daughterboard's J6 header.

Screw the faceplate for the grounding jacks to into the front panel at the top (**Fig. 4**). Use a wire nut to connect 7 wires. Solder 6 of these wires to the solder lugs of 6 black banana jacks (Mueller Electric BU-00233-0) as shown in **Figure 10a**. Solder the 7th wire to a micrograbber (Pomona 4233-9) and hook this into the TP3 test point on the upper left of the MMB0 motherboard (**Figure 10b**), which is connected to ground. Mount these analog ground (AGND) jacks to the ground faceplate.

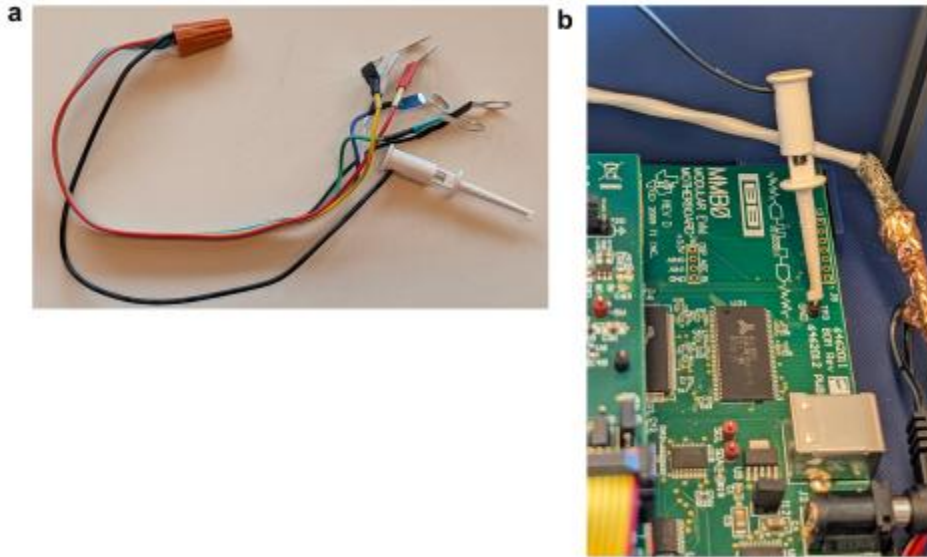


Figure 10. a) Wire nut with 7 wires—6 soldered to solder lugs for the ground input jacks, and one soldered to a micrograbber. **b)** Micrograbber hooked onto the MMB0 motherboard's TP3 test point, which is grounded.

At this point everything should look like **Figure 1b** and OpenSABR is almost complete. Attach the lid with M3 screws at the corners to finish the enclosure.

Custom cabling

The last step is making the custom electrode cable as described in the paper. Solder a minigrabber (Pomona 3925) to the end of each wire of a shielded 3-conductor cable about 90 cm long (**Fig. 11a**). The micrograbbers will hold onto the stripped end of the Left, Right, and Vertex electrode wires. We recommend labeling the micrograbbers accordingly.

Cut 3 short lengths of two-conductor shielded cable for the LR, LV, and RV channels. On the loose end of the 3-conductor electrode cable, split the 3 electrode wires out into the 6 channel wires according to what each channel measures as shown in **Figure 11b**. For example, the R electrode wire should be split into the negative wire of LR and positive wire of RV. After soldering, add some foil or extra shielding to make sure that the shielding is continuous across this cross-over point.

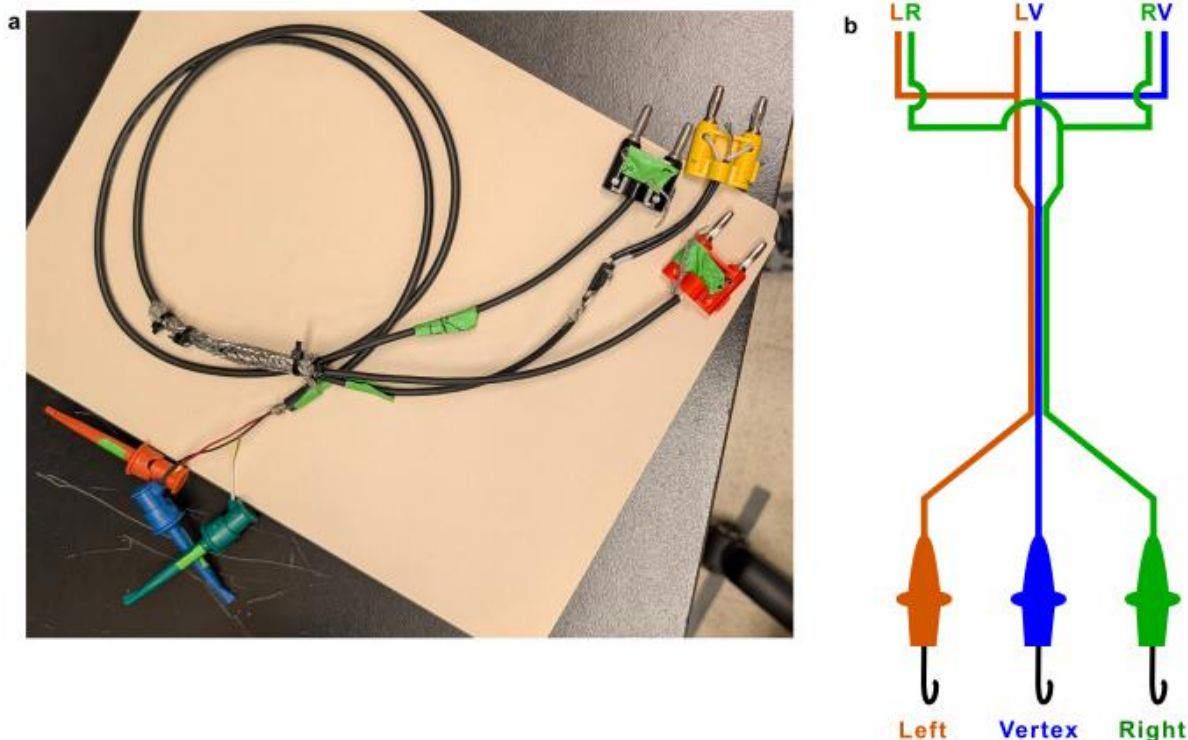


Figure 11. a) Photo of electrode cable with 3 micrograbbers on one end and 3 double banana plugs on the other. Zip-tied shielding sleeve covers the area where the 3-conductor cable splits into the 6 wires for 3 channels, and the shielding sleeve contacts the shielding for all of the individual cables. **b)** Schematic of cable wiring, color-coded by which electrode they connect to.

Description of the Teensy software

The Teensy 4.0 is a 32-bit Arduino-compatible microprocessor board running at 600MHz. It was programmed using the C++ language in the Arduino IDE. The Teensy controls the ADS1299 and acts as an intermediary between the ADS1299 and a computer program, accepting commands from the computer and facilitating high speed data transfer for display, analysis and logging. We have written such programs in National Instruments' Labview and in Python for a Linux PC. The Teensy's code and Linux-based Python GUI are publicly available on our github page (<https://github.com/Rodgers-PAC-Lab/OpenSABR>). There are several ways to upload the program to the Teensy, including from the Arduino IDE or from VSCode using the PlatformIO plugin.

The Teensy interfaces with the ADS1299 via a standard 4-wire SPI bus including chip-select (CS), serial clock (SCLK), Teensy data out (MOSI/PICO) and Teensy data in (MISO/POCI). Other ADS1299 pins required for the interface are START, RESET and data ready (DRDY).

The ADS1299 power-up sequence requires setting chip-select active (CS) then activating RESET for at least 9 μ s before releasing CS. Before starting a recording, you can change the ADS1299 control registers to set the desired sampling rate and gain. Sampling rate will be the same for all channels, but each channel can have its own gain from the allowed values (1, 2, 4, 6, 8, 12, 24). We use a gain of 24 for neural channels and a gain of 1 for the audio channel.

A sampling session begins by sending the read data continuous (RDATAC) and START commands. The falling edge of DRDY signals that the latest sample is ready for reading and triggers an interrupt routine to store the new data. For example, at the highest sampling rate of 16KHz, DRDY falls every 62.5 μ s. Teensy software utilizes an interrupt-driven double-buffered sampling method. All 8 channels are read at once and stored to one of the two buffers. When one buffer is full, the other buffer becomes the active buffer and the full buffer is sent over USB to the computer program. This allows data to be captured and processed continuously without dropping samples.

One 8-channel sample from the ADS1299 includes 3 status bytes plus 3 data bytes/channel, for a total of 27 bytes. Our buffers, or packets, hold 500 samples. Before sending a packet, we prepend a 12-byte header and promote samples from 24 bits to 32-bit integers, making the packet size 16,012 bytes. At 16KHz sample rate a buffer fills in 31.25 ms, for a data rate of 512 Kbytes/sec. The Teensy's USB 2.0 full speed interface has a typical maximum data rate of 30-40 Mbytes/sec, so we are well within the USB bus capability.

The packet header contains fields for packet sequence number, number of samples and channels, sampling rate, and an input trigger marker. The sequence number allows the receiving program to detect lost packets. The trigger marker is a number between 0 and 499 that marks the packet sample on which an external trigger was detected, allowing the receiving program to calculate the sample at which the trigger occurred. Currently this trigger is unused.

ADS1299 provides auxiliary functions for measuring electrode impedance and lead-off detection. By setting the appropriate control registers, an internal current source can be routed to the electrodes. Electrode impedance can be determined from the resulting voltage at the electrode. Similarly, an out of range electrode impedance can be used to detect a lead-off condition.

Transfer of data from the Teensy to the high-level program has been explained above. It uses a binary format due to the high data rate required. On the other hand, commands from the high-level program to Teensy involve a single letter to denote the command and a few numeric parameters. Simple ASCII strings suffice for this and simplify decoding on the Teensy end. Commands supported are:

U - Set up sampling. Parameters: number of channels, sample rate, flags and gain of each channel

S - Start sampling, with parameters previously given by U

I - Inquire if receiver is present. Used at program startup to make sure the Teensy and ADS1299 are operational.

Q - Query specifications. Returns the capabilities of the ADS1299, such as max sample rate, max number of channels, etc.

X - Stop sampling

After sending the Start command, the high-level program polls the USB bus to see if data is available. If so, it reads the packet header and then the raw data. The Stop command is sent when the session completes.